Correction Examen Programmation Java (passé le lundi 29/06/2020)

Exercice1 :

Question 1.1 :

Code Source donné	Ligne d'erreur	Explication

1 class Exercice1 {
2 public static void main(String[] args)
{
3 A x = new A();
4 B y = new B();
5 C z = new C();
6 y.b = 2;
7 z.c = 3;
8 }}
9 abstract class A {
10 int a;}
11 class B extends A {
12 int b;
13 public B(int b){
14 this.b = b;}
15 }
16 class C extends A {
17 final double c = 1;
18 }
19 abstract class D extends A {
20 double d;
21 int operation(int a) {
22 return (a * 2);
23 }
24 abstract int calcul(int b) { }
25 abstract void afficher(); }
	Ligne 3
	
	Ligne 4
	
	Ligne 7
	
	Ligne 14
	
	Ligne 24
		La classe A est une classe abstraite ; une classe abstraite ne peut pas être instancié
		
		Appel d'un constructeur non définit ; le constructeur de la classe B est un constructeur avec paramètre (int)
		
		L'attribut « c » est une constante (déclarée avec final) ; la valeur de c ne peut pas être modifiée
		
		La classe B hérite de A, donc la première ligne du constructeur de la classe fille commence par « super ()».
		
		La méthode calcul est une méthode abstraite donc ne doit pas implémenter ; remplacer « {} » par « ; »

Question 1.2 :

```
public class DateException extends Exception {

	public DateException() {
		super();
	}
	public DateException(String message) {
		super(message);
	}
}
public class DateDefinition {
```

```
	int jour;
	int mois;
	int annee;

	public DateDefinition(int jour, int mois, int annee) throws
DateException {

		if(jour <=0 || jour >=31)
		{
			throw new DateException("le jour ne corresponds
pas");
		}

		this.jour = jour;

		if(mois <=0 || mois >=13)
		{
			throw new DateException("le mois ne corresponds
pas");
		}

		this.mois = mois;

		this.annee = annee;
	}
    // méthode ajoutée pour le test dans main : non demandé dans
l'exercice
	public String toString()
	{
		return ("la date est :" + jour + "/" + mois +  "/"  +
annee);
	}
}
//classe de test non demandé dans l'examen
public class TestDate {

	public static void main(String[] args) {
		// TODO Auto-generated method stub

		try{
		DateDefinition Dt = new DateDefinition(02, 15, 2020);
			System.out.println(Dt);
		}
		catch (DateException e)
		{
			System.out.println(e.getMessage());
		}
	}
```

```
}
```

Problème :

```
public interface ICollaborateur {

	boolean equals (Object o);
	boolean moinsCher(Collaborateur c, int n);

}

public abstract class Collaborateur implements ICollaborateur {

	private int matricule;
	private String nom;
	private double coutJounalier;


	public Collaborateur(int matricule, String nom, double
coutJounalier) {
		this.matricule = matricule;
		this.nom = nom;
		this.coutJounalier = coutJounalier;
	}
	public double getCoutJournalier()
	{
		return coutJounalier;
	}
	public void afficher()
	{
		System.out.println(this.matricule +" " +this.nom + " " +
this.coutJounalier);
	}
	@Override
	public boolean equals(Object Col)
	{
	return  (this.matricule == ((Collaborateur)Col).matricule);
	}

	@Override
	public boolean moinsCher(Collaborateur c, int n) {
		// TODO Auto-generated method stub
		return (this.coutJounalier >  c.getCoutJournalier() );
	}


	@Override
	public String toString() {
		return "Collaborateur [matricule=" + matricule + ", nom="
+ nom + ", coutJounalier=" + coutJounalier + "]";
```

```
	}
	public abstract double calculCoutProjet (int n);

}


public class Consultant extends Collaborateur {

	private double coutProjetForfaitaire;

	public Consultant (int matricule, String nom, double
coutJounalier, double coutProjetForfaitaire)
	{
		super(matricule, nom, coutJounalier);
		this.coutProjetForfaitaire = coutProjetForfaitaire;
	}

	@Override
	public double calculCoutProjet(int n) {
		// TODO Auto-generated method stub
		return ( n * getCoutJournalier() + coutProjetForfaitaire);
	}
	public void afficher()
	{
	  super.afficher();
	  System.out.println("le cout Forfaitaire est: " +
coutProjetForfaitaire +
			"le cout du projet pour 10 jours" +
calculCoutProjet(10));
	}

}

import java.util.ArrayList;

public class ServiceInfo {

	private ArrayList<Collaborateur> lesCollaborateurs;

	public ServiceInfo()
	{
		lesCollaborateurs =  new ArrayList<Collaborateur>();
	}

	public boolean ajouterCollaborateur(Collaborateur Col)
	{
		if(lesCollaborateurs.contains(Col))
		{
			return false;
```

```
            }
            else
            {
                  lesCollaborateurs.add(Col);
                  return true;
            }
      }
      public Collaborateur retirerCollaborateurMoinsCher()
      {
            double coutmin =
lesCollaborateurs.get(0).getCoutJournalier();
            Collaborateur CollaborateurMoinsCher =
lesCollaborateurs.get(0);
            for(Collaborateur Col : lesCollaborateurs)
            {
                  if(Col.getCoutJournalier() < coutmin)
                  {
                        coutmin  =  Col.getCoutJournalier();
                        CollaborateurMoinsCher = Col;
                  }
            }
            lesCollaborateurs.remove(CollaborateurMoinsCher);
            return CollaborateurMoinsCher;
      }
      public ArrayList<Collaborateur> triCroissantParCout ()
      {
            ArrayList<Collaborateur> LesCollaborateursTriee = new
ArrayList<Collaborateur>();
            int i =0;
            while(i < lesCollaborateurs.size())
            {

      LesCollaborateursTriee.add(retirerCollaborateurMoinsCher());
            }
            return LesCollaborateursTriee;
      }

      @Override
      public String toString() {
            return "ServiceInfo [lesCollaborateurs=" +
lesCollaborateurs + "]";
      }

}


public class TestCollaborateur {

      public static void main(String[] args) {
            // TODO Auto-generated method stub
```

```
            ServiceInfo Sinfo1 =  new ServiceInfo();
/*la classe SalariéEntreprise n’est pas développée ce qui explique
la ligne de commentaire ci-dessous */
            //SalariéEntreprise c1 = new SlariéEntreprise(2,"Ben
salah", 60, "concepteur");
            Collaborateur C2 = new Consultant(1, "Ben mrad", 50, 100);
            Collaborateur C3 = new Consultant(2, "Ben Ahmed", 20,
100);
            Collaborateur C4 = new Consultant(3, "Ben Cherif", 80,
100);
            Collaborateur C5 = new Consultant(4, "Ben Mohamed", 70,
100);

            Sinfo1.ajouterCollaborateur(C2);
            Sinfo1.ajouterCollaborateur(C3);
            Sinfo1.ajouterCollaborateur(C4);
            Sinfo1.ajouterCollaborateur(C5);
            System.out.println(Sinfo1);
            ArrayList<Collaborateur> ColTrie =
Sinfo1.triCroissantParCout();
            System.out.println(ColTrie);
      }

}
```